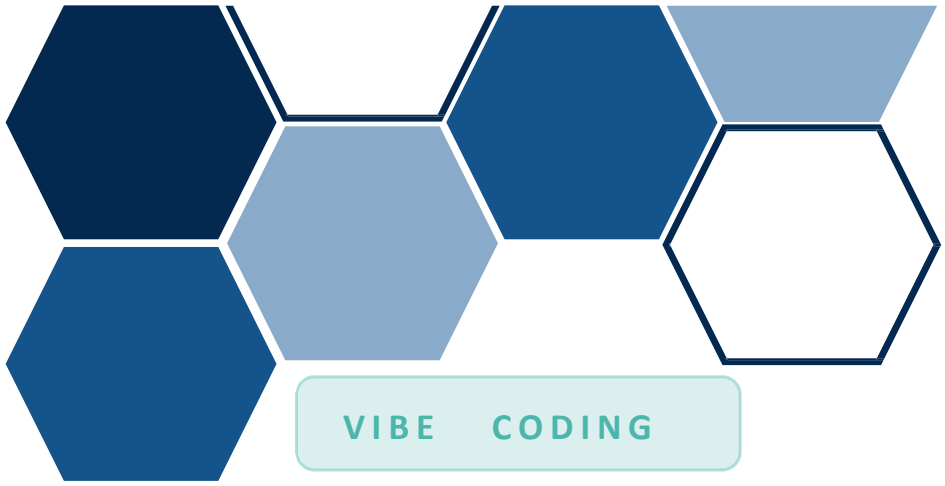


生成式软件开发-6

- Agent Skills

大纲

01 Skills 简介

背景、价值、核心概念与架构层级

02 Skill 的组成与工作机制

目录结构、SKILL.md 格式、渐进式披露

03 入门实践：roll-dice skill

创建、试运行与验证三阶段

04 Description 的优化

触发机制、优化原则与系统化评测

05 最佳实践

Skill 设计与使用的关键原则

06 使用脚本

扩展 Skill 的可执行能力

核心目标：构建可复用的 AI 能力包

01

Skills简介

理解 Skills 的定位、价值与核心概念

背景：为什么需要 Skills

真实工作场景中的问题

大模型可以认为是打包了大量的通用的知识。大模型驱动的 agent/tools 能处理更多类型任务，但在"真实工作"场景里常见问题是：**缺少足够上下文与可复用的过程知识**，导致输出不稳定、不可审计、难以复现。

Skills 的核心定位

用一个简单、开放的文件格式，把**"可复用的专业知识 + 工作流"**打包成可被 agent 按需加载的能力。

场景一

你是一个设计师，积累了很多设计经验。如何传达给大模型让他设计出你的风格的界面？ 设计师如何传达设计经验给大模型，让其设计符合特定风格的界面？

场景二

你是一个有丰富经验的项目经理，有大量的项目管理经验。如何传达给大模型让他根据你的经验来评审项目甚至协助项目组成员展开项目的细节？

Demo

利用 front-design skill 构建一个景点展示页面



Skill 的价值与参与角色

核心价值

1. **提供过程性知识（procedural knowledge）**
   不仅告诉 它"做什么"，更明确"怎么做"
2. **沉淀组织/团队/个人的技能**
   例如工程约定、工具链、内部规范、非显而易见的坑点
3. **可移植、可版本控制**
   skills 就是文件夹，可在多个 agent 产品之间复用，便于审计与持续改进
4. **把多步任务变为可重复工作流**
   形成一致且可追踪的执行路径

参与角色

作者

一次编写，多处部署。将专业知识封装为可复用的能力包。

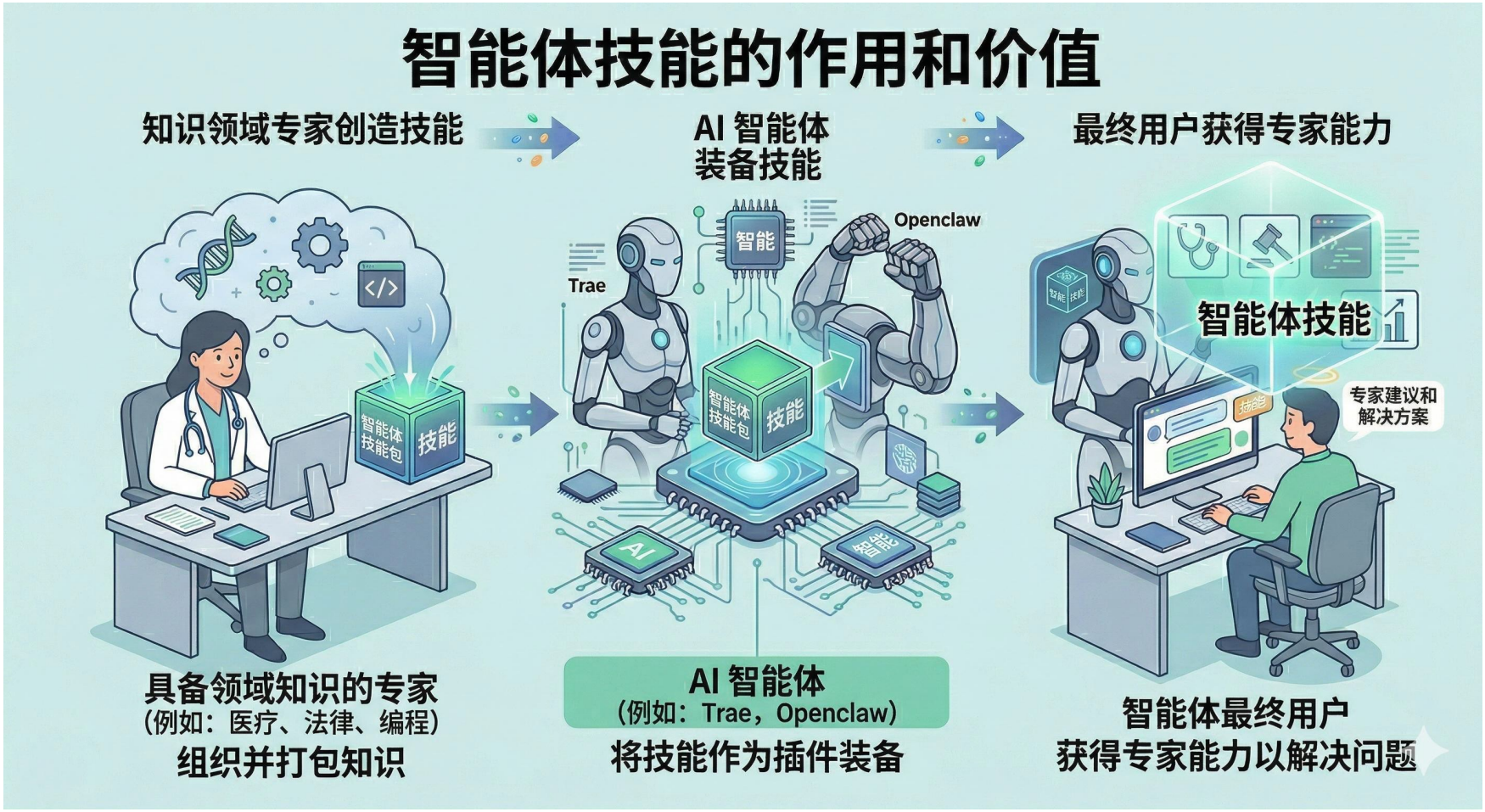
支持 skills 的 agent 产品

用户能"开箱即用"扩展能力，按需加载并执行 skills。

使用者

把组织知识封装成可移植包，纳入版本控制与协作流程。

Skill 的价值与参与角色



Skill 的安装

Agent Skill Installation Guide

安装 Agent Skill

三种安装方式详解：全局安装、项目安装与CLI工具管理

Installation Scope
全局 / 项目 / CLI

Version Control
Git 集成支持

Documentation
技术参考文档

Skill 的安装：基础安装方式

Skill 的安装本质是将包含 SKILL.md 的技能文件夹放置在 Agent 宿主能够发现和加载的特定目录中

全局安装

Global / User Level

适用场景

通用个人生产力工具或偏好，如外语翻译、一键画图、API 测试脚手架等

安装路径

```
~/.trae/skills/
```

核心优势

- 跨项目随时唤起使用
- 无需重复配置

项目安装

Workspace / Project Level

适用场景

与当前项目或特定业务相关，如代码规范检查、部署脚本、业务图表生成等

安装路径

```
.trae/skills/
```

核心优势

- 可随代码库提交到 Git
- 团队成员共享使用
- 确保工作流一致性

选择建议： 个人通用工具选择全局安装，团队协作相关技能选择项目安装，确保技能与使用场景精准匹配

CLI Tool Management

使用 CLI 工具管理 Skill

通过 npx skills 命令实现类似包管理器的规范化工作流

核心命令

ADD 按需安装

npx skills add /

例：npx skills add anthropics/skills/pptx

install 恢复环境

npx skills experimental_install

根据锁定文件恢复一致的技能环境

版本锁定机制

锁定文件

skills-lock.json

记录内容

- Skill来源（source）
- 版本哈希值（version hash）

使用CLI工具的好处

01 按需安装

精准引入当前项目所需技能，避免引入冗余技能，保持项目轻量化和专注性

02 版本锁定

通过锁定文件保证多人在不同环境下执行的确定性，避免版本不一致导致的问题

03 标准化管理

提供标准化的更新和卸载机制，支持多数据源，实现类似 npm 的包管理体验

最佳实践： 正式项目推荐使用 CLI 工具管理 Skill，实现版本控制、团队协作和规范化工作流的完美结合

Demo

- 安装/重装一个处理ppt的skill



核心概念与术语

Agent

能够基于对话、工具、环境执行任务的智能体（具体实现可不同）。

Skill

一个文件夹（能力包），至少包含一个 `SKILL.md`，用于扩展 agent 能力与专业度。

SKILL.md

skill 的入口文件，包含 YAML frontmatter（元数据）与 Markdown 指令正文（执行说明）。

Metadata（元数据）

例如 `name`、`description`，用于"发现"和"激活"阶段的匹配。

渐进式披露（Progressive disclosure）

只在需要时加载更多上下文，避免把所有 skills 全量塞入上下文窗口。

技能使用三阶段

Discovery（发现）→ Activation（激活）→ Execution（执行）

引用 / 脚本 / 资源（References / Scripts / Assets）

可选目录，用于把说明与资源拆分，做到“按需加载”。

架构层级视角：Skill 是 Agent 的"应用插件"

基础工具（Tools）

例如文件读写、终端执行、网页搜索等，属于**原子化的底层功能接口**。

技能包（Skills）

是对基础工具、提示词、业务逻辑和领域知识的**结构化封装**。本质上相当于 Agent 系统的**应用插件**。

Agent 运行时 / 宿主（Runtime/Host）

负责**发现、加载和执行**这些 Skills。将通用的底层能力转化为特定领域的专业工作流。

关键特性： Skill 是**可移植**的——同一个 Skill 理论上可以无缝部署到不同的宿主环境中。

Skill 与其它概念的关系

核心判据： 是否会持续占用上下文窗口

Prompt

特性

一次性提示，仅在一个会话中占用上下文窗口

适用场景

适合一次性问题或探索性沟通，成本低，但难以保证跨会话的一致性

VS. Skill

是**可复用指令包**：适合会反复出现的任务。**经验性信号**：如果同一类 prompt 你已经**重复输入**多次，就应考虑沉淀为 **SKILL.md**

Rules

特性

常驻加载，总是占用上下文窗口

适用场景

常驻的轻量偏好与约束，适合全局一致的"风格/原则"

VS. Skill

是任务/工作流特定的结构化指令，按需触发。 **经验性信号**：当 Rules 文件开始出现大量**“分步骤流程”** **“长模板”** **“特定任务的操作手册”**时，应迁移到 skill，以避免持续占用上下文窗口。

Agentic IDE上下文管理

特性

可以认为是一个内置的Skill

适用场景

根据用户问题动态检索并加载工作目录中的相关代码。这种内置的加载逻辑，本质上就是一个**特殊的、系统级的预置 Skill**。它封装了底层工具（文件读取、向量搜索等），同样遵循“按需加载”的原则，扩展了大模型对当前代码库的感知能力。

VS. Skill

是可扩展的**按需加载**，更节省 tokens、更利于聚焦任务，当你关心**“输出结构固定”** **“步骤固定”** **“边界条件固定”**时，skill 的收益更明显。

选择建议：什么时候优先用 Skill

需求特征	更适合的机制
一次性提问 / 探索性沟通	Prompt
全局 / 项目的风格偏好、轻量约束（语言、格式、习惯）	Rules
需要封装特定领域或专家经验、可复用的工作流、需要一致输出、需要步骤可追踪	Skills

- **输出结构固定**
- **步骤固定**
- **边界条件固定**

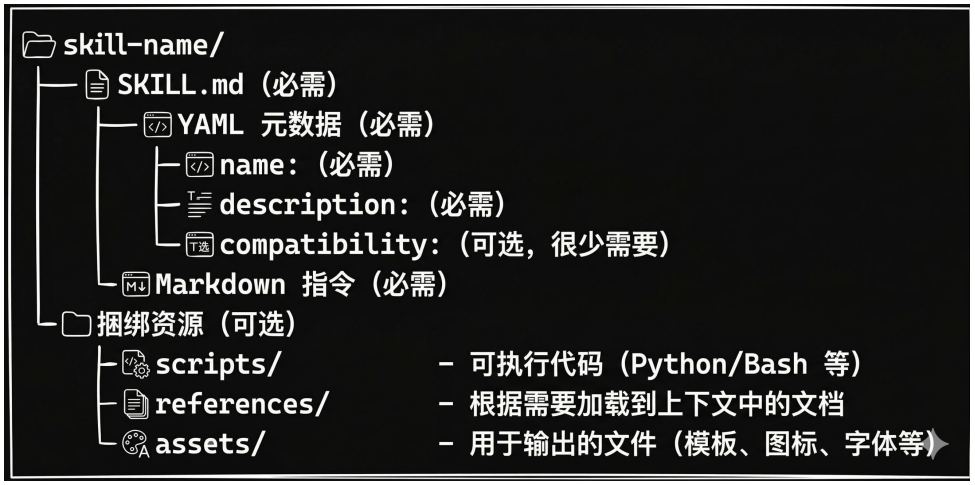
02

Skill 的组成以及工作机制

深入了解 Skill 的目录结构与渐进式披露机制

Skill 的目录结构：一个"可移植能力包"

在 Skills 的开放标准中，一个 skill 本质上是一个目录，**至少**包含 SKILL.md ：



设计动机：技能以文件形式存在，天然适合 Git 管理（版本、评审、回滚、对比）。

SKILL.md

必需

让技能"自描述"（便于审计、改进与分享）。必须包含 YAML frontmatter（元数据）与 Markdown 指令正文（执行说明）。

scripts/

可选

可执行脚本（Python/Bash/JS 等，视宿主 agent 而定）。使 skill 从"纯文本建议"升级到"可执行工作流"。

references/

可选

参考文档（按需读取）。用于把说明与资源拆分，做到"按需加载"。

assets/

可选

模板、图片、资源文件等。支持 skill 的可视化和模板化能力。

SKILL.md：格式、字段与约束

`SKILL.md` **必须包含：**

1. **YAML frontmatter**（位于文件顶部，`---` 包裹）
2. **Markdown 正文**（不限格式，但建议结构化、步骤化）

SKILL.md 的frontmatter的最小示例

```yaml
---
name:
  skill-name
description:
  A description of what this skill does
  and when to use it.
---
```

规范要点

name

必需

- 1 - 64 字符
- 仅小写字母/数字/连字符
- 必须与目录名一致

description

必需

- 1 - 1024 字符
- 描述"做什么 + 何时使用"
- 应包含关键词

license

可选

可写许可证名称或引用 skill 内的许可证文件

compatibility

可选

1 - 500 字符；明确环境要求（语言版本、系统包等）

metadata

可选

任意键值对（string → string），用于扩展属性

SKILL.md：格式、字段与约束

SKILL.md 的frontmatter的完整示例

```
---
name: example-skill
description:
  这是一个示例技能，展示了SKILL.md文件的完整字段用法
compatibility:
  python: ">=3.9"
  dependencies:
    - requests
-     beautifulsoup4
memory_required: "512MB"
metadata:
  version: "2.0.1"
  author: "Developer Name"
  maintainer: "Team Name"
  created_at: "2024-01-01"
  tags: ["example", "demo", "tutorial"]
  status: "stable"
  documentation_url: https://example.com/docs
license: "Apache-2.0"

---
```

规范要点

name

必需

- 1 - 64 字符
- 仅小写字母/数字/连字符
- 必须与目录名一致

description

必需

- 1 - 1024 字符
- 描述"做什么 + 何时使用"
- 应包含关键词

license

可选

可写许可证名称或引用 skill 内的许可证文件

compatibility

可选

1 - 500 字符；明确环境要求（语言版本、系统包等）

metadata

可选

任意键值对（string → string），用于扩展属性

description 的写法（为何重要）

在渐进式披露机制下，agent 在启动/扫描阶段通常 **只加载每个 skill 的 name 与 description**。因此 description 是判定"是否激活"的关键输入。

好的描述：能力 + 触发 + 关键词

```
description:
Extracts text and tables from PDF files, fills PDF forms, and merges
multiple PDFs. Use when working with PDF documents or when the
user mentions PDFs, forms, or document extraction.
```

- ✓ 明确能力范围（提取文本、填充表单、合并 PDF）
- ✓ 明确触发场景（处理 PDF 文档时）
- ✓ 包含关键词（PDFs, forms, document extraction）

差的描述：过于笼统

```
description:
Helps with PDFs.
```

- ✗ 过于简短，难以精确匹配
- ✗ 没有明确能力范围
- ✗ 缺少触发场景和关键词
- ✗ 容易导致"该触发时不触发"或"误触发"

Skill的工作机制：渐进式披露机制

Skills 的核心机制是**分层加载上下文**，通常可理解为三层：

1 层

Metadata

对所有 skills **仅加载 name + description**，用于"知道有哪些技能"。

约百级 token

2 层

Instructions

当匹配到任务时，**加载该 skill 的 SKILL.md 正文**。

建议 < 5000 token

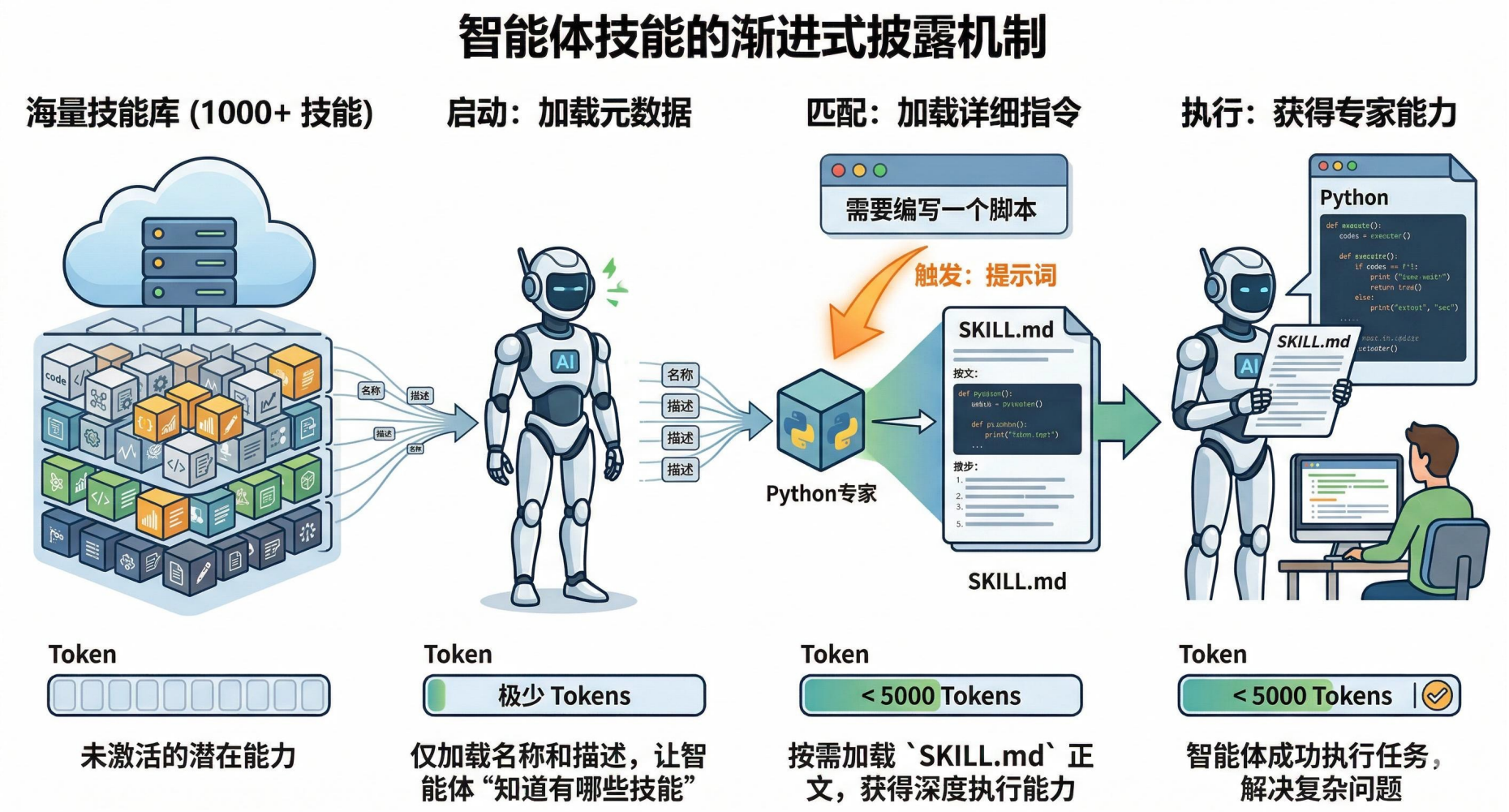
3 层

Resources

脚本 / 参考文档（references）/ 资源文件在特定条件下再加载。

按需

Skill的工作机制：渐进式披露机制



03

入门实践：roll-dice skill

通过实际案例理解 Skill 的创建与使用



创建与试运行 roll-dice skill

SKILL.md 示例（精简版）

```
---
name:
  roll-dice
description:
  Roll dice using a random number generator. Use when
  asked to roll a die (d6, d20, etc.), roll dice, or generate
  a random dice roll.
---
To roll a die, use the following command that generates
a random number from 1 to the given number of sides:
echo $((RANDOM % + 1))
Replace ` <sides> ` with the number of sides on the die
(e.g., 6 for a standard die, 20 for a d20).
```

试运行与验证

在 TRAE 等环境中，使用 Skills 有**两种典型方式：**

1 显式调用

在提示中直接指定要用哪个 skill，例如"Use the roll-dice skill …"。当你希望行为可预测、步骤严格一致时，优先用显式调用。

2 隐式调用

让系统根据任务与 skill 的"何时使用/触发条件"自动选择并加载相关 skill。

底层对应到三阶段： Discovery（识别可用 skills）→ Activation（加载正文）→ Execution（按指令执行）

04 Description的优化

Description 的优化

掌握触发机制与描述优化技巧

Skill的触发机制

◎触发机制的要点

Skill 只有被正确激活才有价值；而在渐进式披露机制下，**description 基本承担了触发的全部信息负担**。

- ✓ 描述过窄会导致"该触发时不触发"
- ✓ 描述过宽会导致"误触发"
- ✓ 启动/扫描阶段通常只加载 name 和 description
- ✓ 当用户任务与 description 匹配时，才会加载完整 SKILL.md

重要细节

很多 agent 实现会优先把 skills 用在"**需要额外知识/能力**"的任务上；非常简单的一步任务，即使匹配，也可能不触发。

写好descriptio的原则

写好 description 的原则

1 使用指令式表述

建议以**"Use this skill when …"**为核心句式，而不是"this skill does …"。

2 聚焦用户意图，不写实现细节

用户说的是**"想要什么"**，不是"你内部怎么做"。

3 更积极的触发覆盖（proactive）

明确列出适用场景，包括**"用户不一定会显式说出关键词"**的情况。

4 保持简洁

规范限制 1024 字符；冗长描述会稀释信号，也会增加全局扫描时的负担。

用"触发评测集"系统化优化

触发评测集

为触发问题设计一组"查询样本"，每条标注是否应触发：

[何时使用本技能]

- "我下载了一个 sales.csv，能帮我画个柱状图吗？"
- "计算一下这份 Excel 表格里每个部门的平均薪资。"

[何时不要使用本技能]

- "怎么把 json 转成 yaml 最快？"（纯文本格式转换）
- "帮我写一段 Python 爬虫代码。"（非本地文件分析）

最佳实践

让 skill 范围合适、指令可执行、成本可控、可迭代

从真实专业知识出发

常见误区

只让模型"凭常识写一个 skill"，而不提供**特定领域的上下文**。这往往会产生空泛表述，缺乏你真正需要的 API 约定、边界条件与项目规范。

推荐来源

1 从一次亲自动手的真实任务中提炼

奏效的步骤、你做过的纠正、输入/输出格式、你提供的上下文

2 从既有工程材料综合

基于真实的事故报告和实际执行过程综合出的"**数据流水线 skill**"，一定会比基于通用"数据工程最佳实践"文章生成的 skill 表现更好

关键在于使用经验或者特定场景的特定的材料

核心原则

真实性 > 通用性

真实的项目经验胜过理论最佳实践

具体性 > 抽象性

API 约定和边界条件比概念描述更重要

实践性 > 理论性

从执行过程中提炼而非从文章中复制

用真实执行来打磨

第一版 skill **很少是最优的**。建议将其投入到真实任务中运行，然后将结果反馈到创建过程中进行迭代优化。



提示： 阅读 Agent 的执行轨迹(traces)，而不仅仅是最终输出

常见问题诊断

指令太模糊
Agent 无法准确理解意图，执行偏离预期

指令不适用于当前任务
skill 描述与实际场景不匹配

太多选项无明确默认值
Agent 在无用的步骤上浪费时间

节省上下文：只写 Agent 缺的内容

一旦 **skill** 被激活，它的完整 **SKILL.md** 正文就会加载到 Agent 的上下文窗口中，与对话历史、系统上下文和其他活动的 skill **争夺注意力**。

核心原则

✓ **专注于 Agent 不知道的内容**

特定于项目的约定、特定于领域的流程、非显而易见的边界情况

✗ **不需要解释基础知识**

PDF 是什么、HTTP 如何工作、数据库迁移的作用

关键问题：“如果没有这条指令，Agent 会做错吗？”如果不会，就删掉它。

节省上下文：示例对比

✕ 反面案例

过于啰嗦 — Agent 早就知道 PDF 是什么

```
## 提取 PDF 文本

PDF（便携式文档格式）是一种包含文本、图像和其他内容的
常见文件格式。

要从 PDF 中提取文本，你需要使用一个库。推荐使用
pdfplumber，因为它能很好地处理大多数情况。
```

⚠ **问题：** 浪费上下文，包含 Agent 已知的常识

✓ 正面案例

直接切入 Agent 靠自己不知道的内容

```
## 提取 PDF 文本

使用 pdfplumber 提取文本。
```

● **优势：** 简洁高效，直接传达关键信息

> 在编写每一条内容时，问问自己："如果没有这条指令，Agent 会做错吗？"如果不会，就删掉它。

把 skill 设计成"内聚单元"

决定一个 skill 应该覆盖什么，就像决定代码里的一个**函数**或**微服务**应该做什么。你希望它封装一个连贯的工作单元，并且能与其他 skill 很好地组合。

核心原则

每个 skill 应该专一地解决一个具体的问题

过窄

把原本连贯的一步拆得太碎，导致 Agent 频繁切换上下文，增加开销并带来指令冲突风险

过宽

把毫无关联的功能塞在一起，导致触发器模糊，难以精确激活，且占用过多无用 token

设计类比

如同函数

封装单一职责，参数清晰，返回值明确

如同微服务

独立部署，松耦合，通过 API 交互

可组合性

与其他 skill 良好协作，构建复杂工作流

复杂任务的解决之道：组合

✕ 反面案例

职责过宽的 Skill

data-master

- 如何连接 MySQL
- 如何清洗缺失值
- 如何使用 matplotlib 画图
- 如何生成 PPT 报告

结果：用户只想画个图，Agent 却把数据库连接的规则也加载了进来，**容易触发模型幻觉**

✓ 正面案例

组合式 Skill

db-querier
专职生成安全高效的 SQL 并查询数据

data-cleaner
专职处理 DataFrame 中的缺失值和异常数据

plot-generator
专职根据清洗后的数据生成美观的可视化图表

优势：每个 skill 职责清晰，按需激活，避免上下文污染

不要试图写一个无所不能的**过宽的 Skill**，而是通过**多个高内聚的 Skill 组合**来完成复杂任务

复杂任务的解决之道：编排与工作流

当你的场景确实需要连续调用多个能力时，与其把所有细节塞进一个文件，不如创建一个**工作流编排 Skill（Workflow Skill）**。

Workflow Skill 的特点

充当"大脑"或"导演"

内部不写具体的执行细节，而是协调多个原子 Skill

定义执行顺序

告诉 Agent 应该按照什么顺序去调用其他 Skill

保持底层工具纯粹性

每个原子 Skill 保持单一职责，通过编排实现复杂流程

示例：会议纪要生成

```
# meeting-report-workflow
```

> 为了生成周报，请依次执行以下步骤：

1. 使用 `lark-calendar` 获取本周会议日程
2. 使用 `lark-vc` 获取会议纪要和逐字稿
   既保持底层工具纯粹性，又实现复杂流程自动化
3. 使用 `data-summarizer` 提取行动项

> 既保持底层工具纯粹性，又实现复杂流程自动化

细节程度要适中

过于全面的问题

过于全面的 skill 往往**适得其反** — Agent 会很难提取真正相关的信息，甚至可能被那些不适用于当前任务的指令误导，从而走上弯路。

推荐做法

经验表明：**简洁、步骤化的指导加上一个可运行的示例**，通常比穷尽所有情况的详尽文档表现更好。

> 当你发现自己在试图覆盖每一个边缘情况时，请考虑：其中大多数情况是否交由 Agent 自己的判断来处理会更好？

平衡的艺术

过度约束

限制 Agent 的灵活性和创造力

恰到好处

清晰步骤 + 可运行示例

过于简略

Agent 缺乏足够指导

对比示例：CSV 数据导出

✕ 反面示例

过度全面的描述

错误示范：过度全面的 Skill 描述

此技能用于将数据导出为 CSV。

执行规则：

1. 如果目标路径已存在同名文件，请先读取该文件，比对文件哈希值...
2. 必须检查输入数据中是否包含逗号，如果包含逗号，必须用双引号包裹...
3. 如果字段中本身包含双引号，必须将双引号转义为两个双引号...
4. 如果数据中包含换行符，必须替换为空格以防止破坏 CSV 结构...
5. 在 Windows 系统下，必须使用 GBK 编码；在 macOS/Linux 下，必须使用 UTF-8...
6. 如果磁盘空间不足 100MB，请立即中止并向用户报错...

问题：过度约束，压抑 Agent 的判断能力

✓ 正面示例

简洁步骤 + 示例

正确示范：简洁步骤 + 示例

此技能用于将结构化数据安全地导出为 CSV 文件。

执行步骤：

1. 接收数据并验证格式。
2. 使用标准库处理数据的转义（逗号、引号等）。
3. 默认使用 UTF-8 编码写入目标路径。遇到同名文件可直接覆盖，除非用户另有要求。

优势：清晰简洁，信任 Agent 的基础能力

为什么过于全面的 Skill 会适得其反

注意力稀释

Attention Dilution

大模型在处理长文本时存在**"中间迷失"**现象

当 Skill 描述中充斥着大量罕见的边缘情况时，Agent **容易忽略核心的主线指令，**导致基本任务执行失败

上下文污染

Context Pollution

如果在 Prompt 中罗列了过多**不适用于当前具体任务**的防范规则

Agent 可能会**过度解读**，将原本简单的正常输入误判为异常情况，从而强行调用不相关的错误处理逻辑

压抑泛化能力

Suppress Generalization

现代大模型自身已经**具备强大的常识推理和基础编码能力**

过度机械化的规则约束会将其降级为**死板的指令执行器**，反而阻碍了其根据具体上下文灵活寻找最优解的能力

信任 Agent 的能力，提供**清晰的指导**而非 **exhaustive 的规则**

用渐进式披露组织大型 Skill

规范建议将 SKILL.md 保持在 **500 行和 5,000 个 token 以内** —— 只保留 Agent 每次运行都需要看到的核心指令。

避免的问题

如果把所有领域知识、API 文档、长篇规范都写在 SKILL.md 中，会迅速消耗 Agent 的上下文窗口，导致 Agent 迷失在海量细节中（**"Lost in the Middle"**现象）。

解决方案

当一个 skill 确实需要更多内容时，请将详细的参考材料移动到 references/ 或类似目录下的独立文件中。

> **关键在于告诉 Agent 何时加载每个文件。**
> "如果 API 返回非 200 状态码，请阅读 `references/api-errors.md` " 比笼统的"查看 references/ 获取详细信息"有用得多。

文件组织建议

SKILL.md

核心指令（< 500 行）
每次运行都必须看到的内容

references/

详细参考材料
按需加载的扩展内容

scripts/

可执行脚本
复杂逻辑的确定性实现

渐进式披露示例：部署助手

SKILL.md 示例

部署助手

你是一个负责处理公司内部项目部署的助手。

部署流程

1. 询问用户要部署的是**前端（Frontend）**还是**后端（Backend）**项目。
2. 根据用户的回答，读取相应的参考文档以获取详细步骤：
   - 如果是前端项目，请读取并应用 `references/frontend-deploy-guide.md`
   - 如果是后端项目，请读取并应用 `references/backend-deploy-guide.md`
3. 严格按照参考文档中的步骤执行部署。

设计亮点

按需加载
只在需要时加载对应的参考文档

明确指令
告诉 Agent 何时读取哪个文件

保持简洁
核心逻辑清晰，细节分离

文件结构

```
deploy-assistant/
├── SKILL.md
└── references/
    ├── frontend-deploy-guide.md
    └── backend-deploy-guide.md
```

校准控制强度

并非 skill 的每个部分都需要相同程度的规定性。你需要将**指令的具体程度**与**任务的脆弱性**相匹配。

匹配具体性与脆弱性

给予 Agent 自由

当**多种方法都有效**且任务能容忍变化时

- 解释**"为什么（why）"**往往比死板的指令更有效

具有规定性

当**操作很脆弱**、一致性很重要，或者必须遵循特定顺序时

- 提供**明确不可更改的命令**

决策框架

关键问题

这个任务是否脆弱？

如果不脆弱

给予自由度，解释原因

如果脆弱

严格规定，明确命令

脆弱性任务 vs 容忍变化的任务

脆弱的任务

必须死板规定

如果 Agent **不严格按照**你给定的格式、顺序或特定参数执行，就会立刻导致报错、系统崩溃或引发严重后果

订单审批流程

订单**必须先审批通过才能付款**，而不是启动订单流程

数据库删除操作

必须**先备份再删除**，且需要二次确认

API 认证流程

必须按照**特定顺序**获取 token 并设置 header

容忍变化的任务

应该给予自由

条条大路通罗马，只要最终结果符合预期，中间怎么实现都行。这类任务往往是创造性的、探索性的

UI 布局设计

使用"杂志风格"布局，而非规定具体的 HTML 结构和像素值

数据分析探索

让 Agent **自主选择**合适的可视化方法和统计检验

代码重构优化

提供目标（如"提高可读性"），让 Agent **自行决定**具体重构策略

提供默认值，而不是菜单

当有多个工具或方法可以工作时，**选择一个默认值**并简要提及替代方案，而不是将它们作为平等的选项呈现。

✕ 反面案例

选项太多

你可以使用 pypdf、pdfplumber、PyMuPDF 或 pdf2image...

问题： Agent 需要额外决策，增加不确定性

✓ 正面案例

清晰的默认值与退路

使用 pdfplumber 提取文本：

```
import pdfplumber
```

对于需要 OCR 的扫描 PDF，改用带有 pytesseract 的 pdf2image。

优势： 明确首选方案，同时提供备选

★ 最佳实践

◎ 明确默认值

选择一个最适合大多数场景的工具

提供退路

简要说明何时使用替代方案

⚡ 减少决策

降低 Agent 的认知负担

Skill范例阅读

阅读：pptx Skill



npx add anthropics/skills@pptx

通过实际案例理解 Skill 书写的最佳实践

Skill范例阅读：pptx

最佳实践原则	在 `pptx` 技能中的具体体现	带来的价值与效果
用渐进式披露组织大型 skill	`SKILL.md` 仅作为快速参考入口。针对复杂的编辑和创建任务，明确指示 Agent 按需阅读 `editing.md` 和 `pptxgenjs.md`。	避免了一次性加载过多细节导致上下文过载，确保 Agent 专注当前任务。
节省上下文：只写 Agent 缺的内容	完全没有解释 PPTX 的通用概念，直接提供 `markitdown` 和 `thumbnail.py` 等具体操作命令和工具路径。	每一行指令都在提供 Agent 依靠自身无法知晓的特定约定，最大化 token 效率。
校准控制强度：提供默认值而非菜单	在设计指南中，直接给出了具体的配色方案（如 Midnight Executive）、特定的字体配对和精确的字号要求（标题 36-44pt）。	减少了 Agent 的猜测空间，直接引导其输出专业级排版，避免 AI 感的同质化设计。
从真实执行经验出发打磨细节	总结了详尽的 `Avoid (Common Mistakes)`，如“永远不要在标题下使用强调线”、“避免文本框边距重叠”等防坑指南。	将过往执行中暴露的问题固化为护栏，直接提升了后续生成的可用性和审美质量。
把验证设计为内聚的闭环	强制要求进行 QA（`Assume there are problems`），并给出了用 `grep` 检查占位符的具体命令，甚至提供了专用于视觉 QA 的 Subagent Prompt。	针对幻灯片生成的脆弱环节施加强控制，确保最终交付前完成自我修正。

使用脚本

通过脚本提升可靠性与确定性

常见的使用场景

当某类逻辑在多次执行中反复出现（解析、验证、生成图表、批处理），把它写成脚本并随 skill 一起发布，通常能显著提升**可靠性与确定性**。

1 复杂的数据处理与转换

例如解析数十万行的 CSV 文件，Agent 直接处理容易出错或被截断，而脚本能保证 **100% 的准确性和效率**

2 外部 API 交互与鉴权

脚本可以封装复杂的鉴权、分页和错误重试逻辑，确保**稳定可靠**的 API 调用

3 确定性的业务规则校验

例如检查配置文件是否符合 JSON Schema。这种**"非黑即白"**的校验应该交给确定性的脚本

4 生成非文本资源

例如根据分析数据生成趋势图表、导出 PDF 报告等，需要**精确的输出格式**

5 保护敏感操作（Guardrails）

对于危险操作，提供一个带有 --dry-run 和安全检查机制的脚本供 Agent 调用

一次性命令

当现成工具即可完成工作、命令不复杂时，可以直接把命令写进 SKILL.md ，不必立刻引入脚本目录。

常见方式

Python

uvx 、 pipx

Node

npx 、 bunx

Deno

deno run

Go

go run

实践要点

尽量固定版本

例如 `npx eslint@9.0.0`，避免随时间漂移

把前置条件写清楚

环境要求写在 `compatibility`，执行步骤写在正文

示例

```
# 使用特定版本的工具
npx prettier@3.0.0 --write .
```

在 SKILL.md 中引用脚本

推荐格式

```
## Available scripts

- scripts/validate.sh — 校验配置
- scripts/process.py — 处理输入数据

## Workflow

1. 运行校验：
   bash scripts/validate.sh "$INPUT_FILE"
2. 处理数据：
   python3 scripts/process.py --input results.json
```

设计原则

先列出可用脚本
让 Agent 知道"有哪些工具可用"

再给出执行步骤
清晰的调用方式和参数说明

路径约定

使用**相对 skill 根目录**的相对路径

自包含脚本

当你希望脚本**"拿来就能跑"**，可采用支持"内联依赖声明"的方式。

Python

推荐：PEP 723 + `uv run`

```
# 依赖声明在脚本头部
# /// script
# dependencies = [
#   "requests>=2.28",
# ]
# ///
```

自动创建隔离环境

Deno

通过 `npm:` / `jsr:` 声明

```
// 直接在 import 中声明
import { serve } from
"jsr:@std/http@0.224";
```

无需 package.json

Bun

自动安装依赖

```
// 适合 TypeScript 快速分发
import { $ } from "bun";
// 首次运行自动安装
```

无需 node_modules

工程建议

版本范围

对依赖尽量加版本范围

锁定策略

必要时引入锁定机制

即用即走

无需预先配置环境

面向 Agent 的脚本接口设计

Agent 运行脚本后主要依赖 **stdout/stderr** 来决定下一步，因此脚本接口要**"机器友好"**。

避免交互式输入

硬性要求

不要让脚本等待 TTY 输入。所有输入应通过**参数、环境变量或 stdin**提供

支持 --help

自我说明

让 Agent 知道怎么用（用途、参数、示例）

错误信息可行动

Actionable Errors

告诉 Agent **哪里错了、期望值是什么、下一步怎么改**

输出尽量结构化

JSON/CSV/TSV

数据输出到 stdout，日志与进度输出到 stderr，避免污染结构化结果

进一步建议

- 支持重试（幂等）
- --dry-run 支持
- 合理的退出码

使用脚本Demo



总结

构建卓越的 Skills

从真实出发，用执行打磨
以简洁为美，让组合发力



Skill Development Best Practices

End

2.氛围编程的关键实践（5）

- Skill系统详解